LangChain Middleware Documentation

Clean, study-friendly version of the supplied documentation.

Documentation Index

Fetch the complete documentation index at: https://docs.langchain.com/llms.txt

Use this file to discover all available pages before exploring further.

Middleware overview

Control and customize agent execution at every step.

Middleware provides a way to more tightly control what happens inside the agent. Middleware is useful for:

- Tracking agent behavior with logging, analytics, and debugging.
- Transforming prompts, tool selection, and output formatting.
- Adding retries, fallbacks, and early termination logic.
- Applying rate limits, guardrails, and PII detection.

Add middleware by passing it to create_agent.

Code: Adding middleware

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware, HumanInTheLoopMiddleware

agent = create_agent(
    model="gpt-5.5",
    tools=[...],
    middleware=[
        SummarizationMiddleware(...),
        HumanInTheLoopMiddleware(...)
    ],
)
```

The agent loop

The core agent loop involves calling a model, letting it choose tools to execute, and then finishing when it calls no more tools.

Middleware exposes hooks before and after each of those steps.

Use middleware inside a LangGraph workflow

Middleware is not a separate runtime: hooks run inside the compiled LangGraph that create_agent returns.

You can drop the whole agent (middleware and all) into a larger StateGraph as a node or subgraph, and every middleware hook continues to run.

Reach for this pattern when the surrounding topology is more than a standard "loop until done": classifying input before routing to one of several agents, fanning out work in parallel, or stitching agent calls together with deterministic steps.

HumanInTheLoopMiddleware matches against each tool's .name.

@tool-decorated functions take their name from the function, so the key below is "send_email".

Code: Middleware inside a LangGraph workflow

```
from langchain.agents import AgentState, create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.graph import START, StateGraph

# Assumes read_email, send_email, classify_node, and route are defined elsewhere.
email_agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[read_email, send_email],
    middleware=[HumanInTheLoopMiddleware(interrupt_on={"send_email": True})],
)

graph = (
    StateGraph(AgentState)
    .add_node("classify", classify_node)
    .add_node("email_agent", email_agent)
    .add_edge(START, "classify")
    .add_conditional_edges("classify", route)
    .compile()
)
```

What travels with the agent

The HITL interrupt, summarization, PII redaction, retries, and any custom hooks all travel with the agent node.

See "Use subgraphs" for the full set of composition patterns, including subgraph checkpointer scoping (per-invocation versus per-thread).

Additional resources

- Built-in middleware — Explore built-in middleware for common use cases.
- Custom middleware — Build your own middleware with hooks and decorators.
- Middleware API reference — Complete API reference for middleware.
- Middleware integrations — Provider-specific middleware for Anthropic, AWS, OpenAI, and more.
- Testing agents — Test your agents with LangSmith.

Resources: https://reference.langchain.com/python/langchain/middleware/

Source links

Connect these docs to your agent of choice via MCP for real-time answers.

Edit this page on GitHub:
https://github.com/langchain-ai/docs/edit/main/src/oss/langchain/middleware/overview.mdx

File an issue: https://github.com/langchain-ai/docs/issues/new/choose